

Data Warehouses

Design and Optimise Analytical Databases

■ DATA WAREHOUSING

■ LEARNING OUTCOME

By the end of this module you will design star and snowflake schemas, implement Slowly Changing Dimensions, optimise queries with partitioning and clustering, and choose between Snowflake, BigQuery, and Redshift.

■ Skills You Will Gain

- Design star and snowflake schemas with fact and dimension tables
- Implement Slowly Changing Dimensions (SCD) Types 1, 2, and 3
- Apply the Medallion architecture: Bronze, Silver, and Gold layers
- Optimise warehouse tables with partitioning, clustering, and materialised views
- Write efficient analytical SQL using window functions and CTEs
- Compare Snowflake, BigQuery, and Redshift for your use case

■ Data warehouses are where business truth lives. Every BI dashboard, executive report, and ML model trains on data engineered in the warehouse. This is the most impactful DE skill.

SECTION 1

Dimensional Modelling

Fact vs Dimension Tables

Fact Table: Contains the measurable events and metrics — orders, clicks, payments, sensor readings. Each row = one business event. Contains foreign keys to dimensions + numeric measures. Dimension Table: Contains the descriptive context — who, what, where, when, why. Customers, products, dates, locations, employees. Grain: The level of detail in the fact table. 'One row per order line item' is finer grain than 'one row per order'.

```
-- Star Schema: fast queries, some redundancy in dimensions
CREATE TABLE fact_sales (
  sale_key BIGINT PRIMARY KEY,
  date_key INT REFERENCES dim_date(date_key),
  customer_key INT REFERENCES dim_customer(customer_key),
  product_key INT REFERENCES dim_product(product_key),
  store_key INT REFERENCES dim_store(store_key),
  -- Measures
  quantity INT,
  unit_price DECIMAL(10,2),
  discount_pct DECIMAL(5,2),
  revenue DECIMAL(10,2),
  cost DECIMAL(10,2),
  profit DECIMAL(10,2)
);
-- Date dimension: pre-built for BI tools
CREATE TABLE dim_date (
  date_key INT PRIMARY KEY,
  -- YYYYMMDD format e.g. 20240115
  date_actual DATE,
  day_of_week VARCHAR(10),
  day_of_month INT,
  month_name VARCHAR(10),
  quarter INT,
  year INT,
  is_weekend BOOLEAN,
  is_holiday BOOLEAN,
  fiscal_quarter INT
);
```

SECTION 2

Slowly Changing Dimensions (SCD)

SCD Type	Behaviour	When to Use	Example
Type 0 — Retain	Never update dimension attributes	Immutable data	Date of birth
Type 1 — Overwrite	Update old value — no history kept	Current state only, history unwanted	Customer phone number
Type 2 — Add Row	Insert new row with date range — full history	Full historical tracking required	Customer address, employee department
Type 3 — Add Column	Add column for previous value only	Need current + one prior value only	Current vs previous region
Type 4 — History Table	Separate history table for changes	Frequent changes to large dimensions	Product pricing history

```
-- SCD Type 2: full history with validity dates
CREATE TABLE dim_customer (
  customer_key INT PRIMARY KEY,
  -- surrogate key (sequence)
  customer_id VARCHAR(20),
  -- business key (from source)
  name VARCHAR(100),
  email VARCHAR(100),
  city VARCHAR(50),
  segment VARCHAR(20),
  valid_from DATE NOT NULL,
  -- when this version became active
  valid_to DATE,
  -- NULL = current active record
  is_current BOOLEAN DEFAULT TRUE
);
-- Query: what city was customer in when they placed order?
SELECT f.order_id, f.revenue, c.city AS customer_city_at_time_of_order
FROM fact_orders f JOIN dim_customer c
ON f.customer_key = c.customer_key
AND f.order_date BETWEEN c.valid_from AND COALESCE(c.valid_to, CURRENT_DATE);
```

SECTION 3

Optimisation Techniques

Technique	How to Apply	Effect
Partitioning	Partition fact tables by date (day/month)	Queries with date filter scan only relevant partitions
Clustering/Sorting	Cluster on frequently filtered columns (region)	Data physically ordered — range scans skip blocks
Materialised Views	Pre-compute heavy aggregations as physical tables	Dashboard queries hit pre-computed result — microsecond

Columnar Storage	Snowflake/BigQuery are columnar by default	Analytics reads only needed columns — 10-100x faster
Result Caching	BigQuery/Snowflake cache identical query results	Repeated same queries cost \$0 and return instantly
Compression	Columnar + encoding compresses 5-10x vs row	Less data scanned = faster queries = lower cost
Statistics	VACUUM ANALYZE (Redshift) / COMPUTE STATISTICS (BigQuery)	Planner picks optimal join and scan strategies

```
-- Snowflake: clustered table for fast date+region queries CREATE TABLE fact_orders CLUSTER BY (DATE_TRUNC('month', order_date), region); -- Materialised view: pre-compute monthly dashboard CREATE MATERIALIZED VIEW mv_monthly_revenue AS SELECT d.year, d.month_name, c.segment, p.category, SUM(f.revenue) AS total_revenue, COUNT(DISTINCT f.sale_key) AS transaction_count, AVG(f.revenue) AS avg_order_value FROM fact_sales f JOIN dim_date d ON f.date_key = d.date_key JOIN dim_customer c ON f.customer_key = c.customer_key JOIN dim_product p ON f.product_key = p.product_key GROUP BY 1, 2, 3, 4; -- Snowflake Time Travel: query historical state SELECT * FROM fact_orders AT (TIMESTAMP => '2024-01-01 00:00:00'::TIMESTAMP_TZ);
```

■ PRACTICE Q & A

Q1. What is the grain of a fact table and why must you define it first?

A: The grain is the level of detail per row — e.g. 'one row per order line item'. Define it first because it determines which dimensions make sense and prevents misaggregating measures. Mixing grains causes double-counting.

Q2. Why use SCD Type 2 instead of just overwriting the customer's city?

A: SCD Type 2 preserves history. You can answer 'what city was this customer in when they placed this order 2 years ago?' — essential for accurate historical analysis and audit requirements.

Q3. What is a surrogate key and why is it preferable to a business key in fact tables?

A: A surrogate key is an auto-generated integer with no business meaning. It's smaller than business keys (VARCHAR), joins faster, and enables SCD Type 2 (multiple rows for same entity with different surrogate keys).

Q4. Why is Snowflake's columnar storage faster for analytics?

A: Analytical queries typically read a few columns from millions of rows. Columnar storage groups each column's data together — only the needed columns are read from disk. Row storage would read all columns unnecessarily.

Q5. What is the Medallion architecture and what problem does it solve?

A: Bronze (raw) → Silver (cleaned) → Gold (business-ready). Solves data quality and governance: you always have the raw source (Bronze), can reprocess from it, and serve clean reliable data (Gold) to analysts and models.

■ Data Warehouse Cheat Sheet	
Star Schema	Fact + denormalised dims – fast queries
Snowflake Schema	Fact + normalised dims – less redundancy
Fact Table	Measures + foreign keys to dimensions
Dimension Table	Descriptive attributes – who/what/where
Surrogate Key	Auto-generated INT PK for dimensions
SCD Type 1	Overwrite – no history
SCD Type 2	New row + valid_from / valid_to
Medallion	Bronze → Silver → Gold layers
Materialised View	Pre-computed physical aggregation
Clustering	Physical sort for faster range scans
Time Travel	Query historical data (Snowflake)
VACUUM ANALYZE	Update statistics for query planner